

PROGRAMACIÓN APLICADA 2026-2

CAPÍTULO 07

Relación 1:N entre módulos en arquitectura hexagonal

Cliente → Vehículo · FK · @ManyToOne · colaboración entre módulos

Documento	Guía del estudiante
Proyecto espejo	ParkFlow 360 · Cliente (1) → Vehículo (N)
Propósito	Replicar la relación 1:N del profesor con las dos entidades equivalentes de su proyecto, sin copiar Cliente/Vehículo.

La FK vive en la base; @ManyToOne vive en persistencia; el dominio conserva la intención del negocio.

1. Qué hará usted en este capítulo

El profesor demostrará la relación Cliente (1) → Vehículo (N) de ParkFlow 360. Usted NO debe crear Cliente/Vehículo salvo que esas sean realmente entidades de su proyecto. Debe utilizar las dos tablas 1:N que seleccionó en capítulos anteriores: una tabla padre y una tabla dependiente que contiene la FK.

REGLA DE TRANSFERENCIA

Copie el patrón arquitectónico, no los nombres del ejemplo. Su dominio, sus atributos, su FK, sus restricciones y sus endpoints deben corresponder al proyecto oficial de su grupo.

2. Complete antes de programar

Dato	Ejemplo del profesor	Su proyecto
Entidad padre	Cliente	_____
PK padre	cliente_id	_____
Entidad dependiente	Vehículo	_____
PK dependiente	vehiculo_id	_____
FK dependiente	cliente_id	_____
Campo UNIQUE	placa	_____
CHECK/enum	tipo	_____
Package base	com.parkflow	com._____

3. Dibuje su relación física

```

<ENTIDAD_PADRE> (1) ----- (N) <ENTIDAD_DEPENDIENTE>
<pk_padre> [PK]               <pk_dependiente> [PK]
                               <fk_padre> [FK]
                               <campo_unico> [UNIQUE]
                               <campo_controlado> [CHECK/ENUM si aplica]
  
```

No avance hasta poder señalar en PostgreSQL exactamente qué columna es la FK y qué constraint la protege.

4. Tres niveles que debe implementar

Nivel	Profesor	Usted debe construir
Dominio	Vehículo tiene clientId	Su entidad dependiente debe conservar el

		id del padre sin usar JPA.
JPA	@ManyToOne ClienteJpaEntity	Su entidad JPA dependiente mapeará la FK.
PostgreSQL	fk_vehiculo_cliente	La FK física que ya diseñó debe seguir intacta.

5. Packages que debe crear

```

com.<suproyecto>.<modulo_dependiente>
├── domain
│   ├── model
│   │   ├── <EntidadDependiente>.java
│   │   └── <EnumSiAplica>.java
│   ├── exception
│   └── port
│       ├── in
│       └── out
├── application
│   └── service
├── infrastructure
│   └── adapter
│       ├── in/web
│       │   ├── dto
│       │   └── mapper
│       └── out/persistence
│           ├── entity
│           ├── mapper
│           └── repository

```

IMPORTANTE

No cree una carpeta global controller/service/repository para todo el sistema. Mantenga cada capacidad dentro de su módulo.

6. Plantilla del enum de dominio

```

package com.<suproyecto>.<modulo>.domain.model;

public enum <NombreEnum> {
    <VALOR_1>,
    <VALOR_2>,
    <VALOR_3>
}

```

Use esta plantilla únicamente si su diseño físico tiene un dominio controlado equivalente. Los valores deben coincidir con su CHECK/regla real.

7. Plantilla de la entidad dependiente de dominio

```

package com.<suproyecto>.<modulo_dependiente>.domain.model;

```

```
public class <EntidadDependiente> {
    private Long id;
    private Long <entidadPadre>Id;
    private String <campoUnico>;
    // agregue los demás atributos reales de su tabla

    public <EntidadDependiente>(Long id,
                                Long <entidadPadre>Id,
                                String <campoUnico> /* ... */) {
        this.id = id;
        this.<entidadPadre>Id = <entidadPadre>Id;
        this.<campoUnico> = <campoUnico>;
    }

    public Long getId() { return id; }
    public Long get<EntidadPadre>Id() { return <entidadPadre>Id; }
    public String get<CampoUnico>() { return <campoUnico>; }
}
```

PROHIBIDO

No coloque @Entity, @ManyToOne o @JoinColumn en esta clase de dominio.

8. Port IN: registrar y consultar

```
public interface Registrar<EntidadDependiente>UseCase {
    <EntidadDependiente> registrar(<EntidadDependiente> entidad);
}

public interface Consultar<EntidadDependiente>UseCase {
    Optional<<EntidadDependiente>> buscarPorId(Long id);
    List<<EntidadDependiente>> listarPor<EntidadPadre>(Long <padre>Id);
}
```

9. Port OUT del módulo dependiente

```
public interface <EntidadDependiente>RepositoryPort {
    <EntidadDependiente> guardar(<EntidadDependiente> entidad);
    Optional<<EntidadDependiente>> buscarPorId(Long id);
    List<<EntidadDependiente>> listarPor<EntidadPadre>Id(Long padreId);
    boolean existePor<CampoUnico>(String valor);
}
```

10. Cómo validar que el padre exista

Su caso de uso dependiente debe utilizar una capacidad pública del módulo padre (un Port IN de consulta) y NO su JpaRepository interno.

```
// PATRÓN CORRECTO
<EntidadDependiente>Service
+--> Consultar<EntidadPadre>UseCase

// PATRÓN INCORRECTO
```

```
<EntidadDependiente>Service
X--> SpringData<EntidadPadre>Repository
```

11. Plantilla del Application Service

```
@Service
public class <EntidadDependiente>Service
    implements Registrar<EntidadDependiente>UseCase,
        Consultar<EntidadDependiente>UseCase {

    private final <EntidadDependiente>RepositoryPort repositoryPort;
    private final Consultar<EntidadPadre>UseCase consultarPadreUseCase;

    public <EntidadDependiente>Service(
        <EntidadDependiente>RepositoryPort repositoryPort,
        Consultar<EntidadPadre>UseCase consultarPadreUseCase) {
        this.repositoryPort = repositoryPort;
        this.consultarPadreUseCase = consultarPadreUseCase;
    }

    @Override
    public <EntidadDependiente> registrar(<EntidadDependiente> entidad) {
        if (consultarPadreUseCase.buscarPorId(entidad.get<EntidadPadre>Id()).isEmpty()) {
            throw new <EntidadPadre>NoEncontradaException(entidad.get<EntidadPadre>Id());
        }

        if (repositoryPort.existePor<CampoUnico>(entidad.get<CampoUnico>())) {
            throw new <CampoUnico>DuplicadoException(entidad.get<CampoUnico>());
        }

        return repositoryPort.guardar(entidad);
    }
}
```

Debe adaptar nombres, normalizaciones y reglas a su proyecto. No copie una validación de placa si su campo UNIQUE significa otra cosa.

12. Plantilla JPA para la relación N:1

```
@Entity
@Table(name = "<tabla_dependiente>", schema = "<su_schema>")
public class <EntidadDependiente>JpaEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "<pk_dependiente>")
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "<fk_padre>", nullable = false)
    private <EntidadPadre>JpaEntity padre;

    // demás columnas exactamente según su diseño físico
}
```

QUÉ DEBE JUSTIFICAR

@ManyToOne aparece porque muchas filas dependientes pueden apuntar a una misma fila padre.

@JoinColumn identifica la columna FK física que ya existe en PostgreSQL.

13. Spring Data Repository adaptable

```
public interface SpringData<EntidadDependiente>Repository
    extends JpaRepository<<EntidadDependiente>JpaEntity, Long> {

    boolean existsBy<CampoUnico>(String valor);
    List<<EntidadDependiente>JpaEntity> findByPadre_Id(Long padreId);
}
```

14. Persistence Mapper

Debe existir conversión explícita entre JPA y dominio. La entidad JPA tiene una referencia al padre; el dominio dependiente conserva solo el identificador del padre.

```
public static <EntidadDependiente> toDomain(<EntidadDependiente>JpaEntity e) {
    return new <EntidadDependiente>{
        e.getId(),
        e.getPadre().getId(),
        e.get<CampoUnico>()
        // ...
    };
}
```

15. Persistence Adapter

```
@Component
public class <EntidadDependiente>PersistenceAdapter
    implements <EntidadDependiente>RepositoryPort {

    private final SpringData<EntidadDependiente>Repository repository;
    private final SpringData<EntidadPadre>Repository padreRepository;

    @Override
    public <EntidadDependiente> guardar(<EntidadDependiente> dominio) {
        var padreRef = padreRepository.getReferenceById(dominio.get<EntidadPadre>Id());
        var entity = <EntidadDependiente>PersistenceMapper.toJpa(dominio, padreRef);
        return <EntidadDependiente>PersistenceMapper.toDomain(repository.save(entity));
    }
}
```

La referencia JPA al repositorio del padre queda dentro de infraestructura. Su caso de uso no debe verla.

16. DTO de entrada obligatorio

```
public record Crear<EntidadDependiente>Request(
    @NotNull Long <entidadPadre>Id,
    @NotBlank String <campoUnico>
    // ... campos que recibe el cliente HTTP
)
```

) {}

17. Endpoints mínimos que debe entregar

Operación	Método	Ruta modelo
Registrar dependiente	POST	/api/<dependientes>
Consultar por id	GET	/api/<dependientes>/{id}
Listar por padre	GET	/api/<dependientes>/<padre>/{padreId}

18. Pruebas obligatorias

- Registro válido utilizando un padre existente.
- Consulta por id del registro creado.
- Listado de dependientes del mismo padre.
- Intento de registro con padre inexistente.
- Intento de duplicar el campo protegido por UNIQUE.
- Intento de enviar un valor inválido si existe CHECK/enum.

19. Comprobación SQL obligatoria

```
SELECT d.*, p.*
FROM <schema>.<tabla_dependiente> d
JOIN <schema>.<tabla_padre> p
  ON p.<pk_padre> = d.<fk_padre>
ORDER BY d.<pk_dependiente>;
```

Adapte los nombres. Debe poder señalar visualmente la FK y demostrar que cada fila dependiente apunta a un padre válido.

20. Errores que invalidan la entrega

- Controller inyectando JpaRepository.
- Entidad de dominio anotada con @Entity o @ManyToOne.
- Eliminar la FK de PostgreSQL porque “Java ya valida”.
- Copiar Cliente/Vehiculo aunque no existan en su proyecto.
- Usar nombres genéricos padre/hijo en el código final en vez de nombres del dominio.
- No probar el caso de padre inexistente.
- No poder explicar por qué la relación es 1:N.

21. Evidencia a entregar

- Árbol de packages del módulo dependiente.
- Código de dominio y enum si aplica.
- Ports IN/OUT.
- Application Service.
- JpaEntity con @ManyToOne/@JoinColumn.
- Persistence Adapter y mapper.
- Controller + DTO.
- Resultados HTTP de casos positivos y negativos.
- JOIN ejecutado en DataGrip.
- Commit Git.

```
git add .
git commit -m "feat: implement <módulo-dependiente> relation with <módulo-padre>"
```

22. Preguntas para su defensa individual

- ¿Cuál es su tabla padre y cuál es su tabla dependiente?
- ¿Dónde está su FK física?
- ¿Por qué utiliza @ManyToOne y no @OneToMany en la columna FK?
- ¿Por qué su dominio no tiene anotaciones JPA?
- ¿Cómo valida que el padre exista?
- ¿Por qué no llama al JpaRepository del padre desde su Service?
- ¿Qué restricción UNIQUE posee su tabla dependiente?
- ¿Qué sucede si la base recibe una FK inexistente?
- ¿Qué significa LAZY?
- Explique el recorrido completo desde POST hasta PostgreSQL.

23. Definition of Done

- La relación 1:N de su proyecto está identificada y coincide con PostgreSQL.
- El módulo dependiente posee estructura hexagonal.
- El caso de uso valida el padre mediante un contrato del módulo padre.
- La JpaEntity dependiente mapea correctamente la FK.
- La entidad de dominio está libre de JPA.
- Los endpoints funcionan.
- Los casos negativos funcionan.
- La relación se comprueba con JOIN en DataGrip.
- Todos los integrantes pueden defender la arquitectura.

SIGUIENTE CAPÍTULO

En el Capítulo 08 se profesionalizará el tratamiento de errores: @RestControllerAdvice, respuestas HTTP uniformes, validaciones de negocio y transacciones. No avance si todavía no puede explicar esta relación 1:N

de punta a punta.